Breaking and Improving LLM-Written Specs with Program Synthesis

ECE1724H F3 — Program Synthesis and Verification, Fall 2026 — Project Proposal

Team: Yicheng Yao (`yicheng.yao@mail.utoronto.ca`), Jarvis Wang (`jarvis.wang@mail.utoronto.ca`)

1. Problem and Motivation

Verification proves that a program matches its spec, not that the spec says what we meant. A spec can be **too weak** (accepts wrong programs), **too strict on inputs**, **empty**, or **too strong** (rejects the correct program). Hardware engineers catch similar problems (*vacuous passes*, *over-constrained assumptions*) with cover and vacuity checks, but software verification has no standard equivalent. This matters more as LLMs write specs: an LLM-written spec can look fine yet be too weak, and the usual check against a few tests misses such weaknesses and gives the LLM little feedback.

Our idea is to attack the spec. We turn program synthesis around and synthesize a *wrong* program that still passes the spec: an LLM proposes it and Z3 confirms it. Such a program proves the spec is too weak and explains the problem, so we hand it back to the spec-writing LLM to fix the spec.

2. Approach

2.1 Tasks and specs

Each benchmark task has three hand-written parts: a plain-English **description**; the **allowed inputs** (input types, such as integers or fixed-length arrays, plus any assumptions stated in the description); and a **golden spec** that says which outputs are correct. For tasks with one correct answer, we write a **golden program** and the golden spec is "the result equals the golden program's output". For tasks with many correct answers, we write the golden spec directly. We test each golden artifact on hand-picked cases and review it within the team. These artifacts exist only to measure spec quality in our benchmark; they are never taken from the LLM.

2.2 Checking the spec

Each LLM-written spec, a precondition `pre` and a postcondition `post` written as Z3 formulas, goes through up to five checks. The first four are single Z3 queries; when Z3 finds a satisfying assignment (**sat**), it is a concrete example of the problem, except for the empty-precondition check, where **unsat** is the problem. Here *golden* means the golden spec.

Check	What we give Z3	Z3 returns sat (found)	Z3 returns unsat (none exist)
Empty precondition	allowed inputs **and** `pre`	✅ fine: some allowed input satisfies `pre`	❌ **empty**: no allowed input satisfies `pre`
Too strict on inputs	allowed inputs **and not** `pre`	❌ **too strict**: an allowed input that `pre` rules out	✅ fine
Too strong	allowed inputs **and** `pre` **and** golden accepts `r` **and not** `post(r)`	❌ **too strong**: an input where an intended output is rejected	✅ fine
Too weak (single output)	allowed inputs **and** `pre` **and** `post(r)` **and** golden rejects `r`	❌ **too weak**: an input plus a wrong output the spec accepts	✅ fine
Too weak (whole program)	run the attacker (Section 2.3), only if the single-output check found the spec too weak	—	—

A postcondition that holds for every output (the second kind of empty spec) is caught by the too-weak check.

2.3 The attacker

The attacker looks for a program `p` that (1) passes the LLM's spec on every allowed input, and (2) is wrong on some allowed input, i.e., differs from the golden program or breaks the golden spec. We ask an LLM for the simplest plausible implementation that passes the spec but is wrong, written in the same small expression language as the golden program. The attacker runs as a guess-and-check loop in the style of counterexample-guided synthesis

(CEGIS): we give the LLM the example from the single-output check as a hint, the LLM proposes a program, Z3 checks both conditions, and if either fails we tell the LLM why and let it retry, up to 3 attempts. Since Z3 confirms every program, the attacker never reports a false alarm.

Why an LLM rather than a search-based synthesizer. LLMs write realistic programs without needing a grammar, while classical search grows exponentially with program size and only finds programs its grammar can express. A failed LLM attack proves nothing, but detection does not rely on it: the single-output check already finds every too-weak spec, which also lets us measure the attacker's success rate automatically.

Future work. If time allows, we will build a search-based attacker that enumerates programs from smallest to largest, checks each with Z3 using the same two conditions, and stops at the first program that passes the spec but is wrong, which is then the shortest such program in the grammar.

2.4 Fixing the spec

Repair starts from a first spec that is too weak. In each round (up to 3), we run all checks; if any fails, the LLM rewrites the spec given the description, the old spec and the feedback. We compare three kinds of feedback: (1) **pass/fail only**, our baseline; (2) **a single example** from the failed check; and (3) the same as (2), but a too-weak spec is reported with **a whole wrong program** from the attacker.

3. Evaluation

3.1 Setup

Benchmarks. We start with 30 tasks, 10 in each of three groups, and add more if time allows: small integer functions; integer and array problems adapted from MBPP/HumanEval-style datasets with fixed-length arrays; and tasks with many correct answers, which check that we do not flag specs that are correct but loose. Every golden spec must also pass all of our checks, which serves as a test of the checks themselves.

Experiments. We fix the model version and settings and cache all responses. In **Experiment 1** (RQ1, RQ2, RQ4), the LLM writes a first spec for every task three times; every spec is checked, and every too-weak spec is attacked. In **Experiment 2** (RQ3), we take the too-weak specs from one repeat and run the repair loop once with each feedback type. This is at most about 400 LLM calls. For every spec, attack and repair run, we log the check results, attack outcomes, rounds used, and Z3 and LLM cost.

3.2 Correctness

A spec is **correct** if Z3 proves it logically equivalent to the golden spec, i.e., both accept exactly the same outputs for every allowed input. The four Z3 checks match this definition: a spec that is not equivalent must be too strict, too strong or too weak. So the checks always explain *how* an incorrect spec is wrong, and the repair loop only stops on a correct spec (unless Z3 answers "unknown").

3.3 Research questions

- **RQ1: How common are bad specs?** Defect rates of first specs by task group, averaged over the three repeats, plus the most common missing conditions.
- **RQ2: How good is the attacker?** On specs known to be too weak, the attacker's success rate, attempts and program size; compared with the search-based attacker if built.
- **RQ3: What feedback helps the LLM fix its spec?** The share of repair runs ending with a correct spec and the rounds needed, per feedback type. Since all three start from the same spec, each spec is its own control; the key comparison is a whole wrong program (3) against a single example (2).
- **RQ4: Is it practical?** Time and cost per spec for checks, attacks and LLM calls, and how often Z3 answers "unknown" on array tasks.

4. Discussion

- **Why synthesize a whole wrong program?** Z3 alone can only give one bad point, a single input and wrong output, because it finds values, not programs. A whole program shows the *kind* of mistake the spec allows, which points at the missing condition, shows exactly what could slip through verification, and stays readable on larger tasks. It may also give the LLM more useful feedback for rewriting the spec, which is what RQ3 tests.
- **If a golden program exists, why need a spec?** In real work there is usually no golden program; ours exist only to measure spec quality. In practice, the attacker would show its program to a person, who decides whether that behaviour is intended, much like reviewing a cover trace in hardware verification.
- **Are the tasks realistic enough?** Small tasks are deliberate, since they allow trusted golden specs and exact measurement, and they still cover common causes of weak specs: missing cases, index bounds, quantifiers and many-answer tasks. If time allows, we will audit specs from a public Dafny-based benchmark and add tasks with state and timing.